

Clase 07 — Accesos, usuarios y tienda de ropa con variantes

Materia: Laboratorio de Programación y Lenguajes 2026

Duración: flexible; continúa desde el último checkpoint consistente

Modalidad: tutorial guiado con live coding docente

Estado inicial: `repo-inicial/` ejecutable

Producto final: Tienda v0.1 navegable con productos, roles y tests

Preparación docente

Antes de la clase, descargar o actualizar el repo inicial y verificar conectividad para instalar dependencias. La creación, activación del entorno e instalación se repiten visiblemente durante `[F-03]`; no deben omitirse de la demostración.

No ejecutar `migrate` antes de crear `accounts.User` y configurar `AUTH_USER_MODEL`. Mantener disponibles los checkpoints descritos en `repo-inicial/CHECKPOINTS-DOCENTE.md`.

[F-00] De un theme a una tienda

Qué realiza el profesor: presenta la meta visible: MiniStore dejará de ser HTML estático y terminará como Tienda v0.1 conectada al ORM, con usuarios, permisos y tests.

Qué decir:

- “Hoy no vamos a construir archivos aislados; vamos a cerrar una primera versión.”
- MiniStore aporta presentación; Django aporta comportamiento y reglas.
- Cada bloque termina con una verificación observable.

Transición: mostrar la aplicación final preparada en el checkpoint `C06-v01`.

[F-01] El producto final ya es visible

Qué realiza el profesor: recorre brevemente Tienda v0.1 como visitante, cliente, operador y administrador.

Verificación visible:

- visitante navega catálogo y detalle;
- cliente abre Mi cuenta;
- operador ve productos pero no usuarios;
- administrador ve todo.

Pregunta de control: "¿Qué diferencias corresponden a presentación y cuáles a permisos?"

Transición: volver a `C00-starter` .

[F-02] La arquitectura separa responsabilidades

Qué realiza el profesor: presenta las responsabilidades y dependencias entre apps, y abre el árbol del repo inicial.

Conceptos clave:

- `accounts` será propietaria de identidad;
- `products` será propietaria del catálogo;
- `orders` y `operations` todavía no existen;
- `orders` dependerá de usuarios y variantes;
- `operations` dependerá de las órdenes.

Pregunta de control: "¿Por qué pagos no debería importar directamente modelos de productos?"

Transición: ejecutar el starter antes de modificarlo.

[F-03] Activamos un entorno reproducible

Qué realiza el profesor: entra al repo inicial, crea un entorno aislado, lo activa, instala las dependencias fijadas y verifica el starter.

```
cd repo-inicial
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m django --version
python -m pytest --version
python manage.py check
python -m pytest -q
python manage.py runserver
```

Qué explicar:

- `.venv` aísla las dependencias respecto de otros proyectos;
- la activación selecciona el intérprete y paquetes de la tienda;
- `requirements.txt` fija Django, Jazzmin, Pillow y testing;
- `python -m pip` y `python -m pytest` usan el Python activo;
- `.venv` no se versiona.

Verificación visible:

```
python -c "import sys; print(sys.executable)"
python -m pip show Django django-jazzmin pytest-django Pillow
```

Abre `/` y `/admin/` .

Alternativa Linux/macOS:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Resultado esperado: Python dentro de `.venv` , dependencias instaladas, check sin errores y test starter en verde.

Transición: crear la app que debe existir antes de la primera migración.

[F-04] Creamos la app propietaria de la identidad

Qué realiza el profesor:

```
python manage.py startapp accounts
```

Agrega `"accounts"` después de `"jazzmin"` en `INSTALLED_APPS` .

```
INSTALLED_APPS = [
    "jazzmin",
    "accounts",
    "django.contrib.admin",
    # ...
]
```

Crea `accounts/urls.py` :

```
from django.urls import path
from .views import ProfileView

app_name = "accounts"

urlpatterns = [
    path("profile/", ProfileView.as_view(), name="profile"),
]
```

Error frecuente: ejecutar `migrate` en este punto antes de configurar el usuario custom.

Transición: definir el modelo User.

[F-05] Separamos identidad común y perfil cliente

Qué realiza el profesor: reemplaza `accounts/models.py` .

```

from django.conf import settings
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    pass

class CustomerProfile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        related_name="customer_profile",
        on_delete=models.CASCADE,
    )
    phone = models.CharField("teléfono", max_length=30, blank=True)
    shipping_address = models.TextField("dirección de entrega", blank=True)

    def __str__(self):
        return f"Perfil cliente de {self.user}"

```

Agrega al final de `config/settings.py` :

```

AUTH_USER_MODEL = "accounts.User"
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "accounts:profile"
LOGOUT_REDIRECT_URL = "starter-home"

```

Actualiza `config/urls.py` :

```

from django.urls import include, path

urlpatterns = [
    path("", StarterHomeView.as_view(), name="starter-home"),
    path("accounts/", include("django.contrib.auth.urls")),
    path("accounts/", include("accounts.urls")),
    path("admin/", admin.site.urls),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

```

Ahora sí:

```

python manage.py makemigrations accounts
python manage.py migrate

```

Qué enfatizar:

- cambiar `AUTH_USER_MODEL` tarde es costoso; por eso se define antes de migrar;
- `User` representa identidad, autenticación, grupos y permisos para todos;
- `CustomerProfile` contiene datos exclusivos del cliente;
- operador y administrador no reciben un perfil cliente.

Transición: crear una vista cliente protegida.

[F-06] Mi cuenta confirma la autenticación

Qué realiza el profesor: crea `accounts/views.py` .

```

from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.views.generic import TemplateView

from .models import CustomerProfile

class ProfileView(LoginRequiredMixin, UserPassesTestMixin, TemplateView):
    template_name = "accounts/profile.html"

    def test_func(self):
        return not self.request.user.is_staff

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["customer_profile"], _ = CustomerProfile.objects.get_or_create(
            user=self.request.user
        )
        return context

```

Crea `accounts/templates/accounts/profile.html` :

```

{% extends "store/base.html" %}
{% block title %}Mi cuenta | Tienda v0.1{% endblock %}
{% block content %}
<section class="container py-5">
    <h1>Mi cuenta</h1>
    <p>{{ request.user.get_full_name|default:request.user.username }}</p>
    <p>{{ request.user.email }}</p>
    <p>{{ customer_profile.phone|default:"Sin teléfono registrado" }}</p>
    <p>{{ customer_profile.shipping_address|default:"Sin dirección registrada" }}</p>
</section>
{% endblock %}

```

Crea `accounts/templates/registration/login.html` :

```

{% extends "store/base.html" %}
{% block title %}Ingresar | Tienda v0.1{% endblock %}
{% block content %}
<section class="container py-5">
    <h1>Ingresar</h1>
    <form method="post" class="col-md-6">
        {% csrf_token %}
        {{ form.as_p }}
        <button class="btn btn-dark" type="submit">Ingresar</button>
    </form>
</section>
{% endblock %}

```

Crea `accounts/tests/test_auth.py` :

```
import pytest
from django.urls import reverse

@pytest.mark.django_db
def test_profile_requires_login(client):
    response = client.get(reverse("accounts:profile"))
    assert response.status_code == 302
    assert reverse("login") in response.url

@pytest.mark.django_db
def test_staff_user_cannot_open_customer_profile(client, django_user_model):
    operator = django_user_model.objects.create_user(
        "operator", password="secret123", is_staff=True
    )
    client.force_login(operator)

    response = client.get(reverse("accounts:profile"))

    assert response.status_code == 403
```

```
pytest accounts/tests/test_auth.py -q
```

Transición: distinguir entrar al admin de tener permisos.

[F-07] is_staff abre la puerta, no concede permisos

Qué realiza el profesor: crea tres usuarios desde shell para hacer visible la diferencia.

```
python manage.py createsuperuser
python manage.py shell
```

```
from accounts.models import CustomerProfile, User

client_user = User.objects.create_user("cliente", password="cliente123")
CustomerProfile.objects.create(user=client_user)
operator = User.objects.create_user("operator", password="operator123", is_staff=True)
```

Demostración: solo el cliente posee `CustomerProfile` ; cliente no entra al admin; operador entra pero todavía no ve modelos propios y recibe 403 en Mi cuenta.

Pregunta de control: "¿Qué habilitó `is_staff` y qué no habilitó?"

Transición: expresar el rol mediante un grupo.

[F-08] Creamos el grupo Operadores

Qué realiza el profesor: crea el grupo, aún sin permisos comerciales porque `products` todavía no existe.

```
from django.contrib.auth.models import Group

operator_group, _ = Group.objects.get_or_create(name="Operadores")
operator.groups.add(operator_group)
```

Explica:

- el grupo expresa rol;
- los permisos se asignarán después de migrar `products` ;
- no se agrega un campo `role` duplicado en User.

Transición: comparar los accesos esperados.

[F-09] El mismo admin muestra opciones diferentes

Qué realiza el profesor: abre tres sesiones privadas o perfiles de navegador y compara.

Qué decir:

- Jazzmin aplica el mismo skin.
- Django construye el menú a partir de permisos.
- El operador nunca recibe permisos sobre `accounts.User` , `Group` o `Permission` .
- El cliente no necesita permisos de modelos para usar vistas públicas/propias.

Transición: escribir un test que no dependa de la apariencia del menú.

[F-10] Probamos la URL que el menú no muestra

Qué realiza el profesor: crea `accounts/tests/test_admin_permissions.py` .

```
import pytest
from django.urls import reverse
from accounts.models import User

@pytest.mark.django_db
def test_operator_cannot_open_users(client):
    operator = User.objects.create_user(
        "operador-test", password="secret123", is_staff=True
    )
    client.force_login(operator)

    response = client.get(reverse("admin:accounts_user_changelist"))

    assert response.status_code == 403
```

```
pytest accounts/tests/test_admin_permissions.py -q
```

Qué enfatizar: el test usa URL directa; no confía en que el enlace esté oculto.

Transición: personalizar el panel sin confundirlo con autorización.

[F-11] Jazzmin organiza el panel interno

Qué realiza el profesor: amplía `JAZZMIN_SETTINGS` .

```
JAZZMIN_SETTINGS.update(  
    {  
        "order_with_respect_to": [  
            "products",  
            "accounts",  
            "auth",  
        ],  
        "icons": {  
            "accounts.User": "fas fa-user",  
            "products.Product": "fas fa-shirt",  
            "products.Category": "fas fa-tags",  
        },  
    },  
)
```

Nota docente: las entradas `products.*` aparecerán después de crear esa app.

Transición: registrar correctamente la identidad común.

[F-12] UserAdmin administra identidad y permisos

Qué realiza el profesor: reemplaza `accounts/admin.py` .

```
from django.contrib import admin  
from django.contrib.auth.admin import UserAdmin as DjangoUserAdmin  
from .models import User  
  
@admin.register(User)  
class UserAdmin(DjangoUserAdmin):  
    pass
```

Qué enfatizar: `CustomerProfile` no se registra en Django Admin. El administrador gestiona identidad, `is_staff` , grupos y permisos; teléfono y dirección pertenecen al área cliente.

Verificación: superusuario ve Users sin campos comerciales; operador recibe 403 por URL directa; ninguno posee `CustomerProfile` salvo que sea creado explícitamente como cliente.

Transición: cerrar identidad con una batería breve.

[F-13] El bloque de identidad queda protegido

Qué realiza el profesor:

```
pytest accounts -q
```

Agrega opcionalmente un segundo test preparado:


```
@pytest.mark.django_db
def test_authenticated_customer_opens_profile(client):
    user = User.objects.create_user("ana", password="secret123")
    client.force_login(user)
    response = client.get(reverse("accounts:profile"))
    assert response.status_code == 200
    assert "Mi cuenta" in response.content.decode()
    assert hasattr(user, "customer_profile")
```

Transición: crear la app propietaria del producto.

[F-14] Creamos la app propietaria del catálogo

```
python manage.py startapp products
```

Agrega "products" a INSTALLED_APPS y crea products/urls.py :

```
from django.urls import path
from .views import HomeView, ProductDetailView, ProductListView

app_name = "products"

urlpatterns = [
    path("", HomeView.as_view(), name="home"),
    path("products/", ProductListView.as_view(), name="list"),
    path("products/<slug:slug>/", ProductDetailView.as_view(), name="detail"),
]
```

Transición: separar producto de variante.

[F-15] Una prenda no es una unidad vendible

Qué realiza el profesor: modela verbalmente una remera con tres combinaciones.

Qué decir:

- nombre y descripción pertenecen a Product;
- talla/color/SKU/precio/stock pertenecen a ProductVariant;
- futuras órdenes referenciarán la variante;
- stock en Product produciría ambigüedad.

Transición: escribir los modelos.

[F-16] Modelamos catálogo, imágenes y variantes

Qué realiza el profesor: reemplaza products/models.py .

```

from django.core.validators import MinValueValidator
from django.db import models
from django.urls import reverse

class Category(models.Model):
    name = models.CharField(max_length=80, unique=True)
    slug = models.SlugField(unique=True)

    class Meta:
        verbose_name_plural = "categories"

    def __str__(self):
        return self.name

class Product(models.Model):
    category = models.ForeignKey(Category, related_name="products", on_delete=models.PROTECT)
    name = models.CharField(max_length=160)
    slug = models.SlugField(unique=True)
    brand = models.CharField(max_length=80, blank=True)
    description = models.TextField()
    active = models.BooleanField(default=True)
    featured = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        permissions = [("publish_product", "Puede publicar productos")]
        ordering = ["-created_at"]

    def __str__(self):
        return self.name

    def get_absolute_url(self):
        return reverse("products:detail", kwargs={"slug": self.slug})

    @property
    def starting_price(self):
        variant = self.variants.filter(active=True).order_by("price").first()
        return variant.price if variant else None

class ProductVariant(models.Model):
    product = models.ForeignKey(Product, related_name="variants", on_delete=models.CASCADE)
    size = models.CharField(max_length=10)
    color = models.CharField(max_length=40)
    sku = models.CharField(max_length=40, unique=True)
    price = models.DecimalField(
        max_digits=10, decimal_places=2, validators=[MinValueValidator(0)]
    )
    stock = models.PositiveIntegerField(default=0)
    active = models.BooleanField(default=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["product", "size", "color"],
                name="unique_product_size_color",
            )
        ]

    def __str__(self):
        return f"{self.product} / {self.size} / {self.color}"

    @property
    def available(self):
        return self.active and self.stock > 0

class ProductImage(models.Model):
    product = models.ForeignKey(Product, related_name="images", on_delete=models.CASCADE)
    image = models.ImageField(upload_to="products/")
    alt_text = models.CharField(max_length=160)
    primary = models.BooleanField(default=False)

```

Transición: convertir invariantes en migraciones y tests.

[F-17] Las restricciones convierten reglas en datos válidos

```
python manage.py makemigrations products
python manage.py migrate
```

Qué explica el profesor:

- `PROTECT` evita eliminar una categoría usada;
- `CASCADE` elimina variantes si desaparece el producto;
- SKU y combinación son únicas;
- `PositiveIntegerField` rechaza stock negativo a nivel de validación del modelo/form.

Agrega a `config/urls.py` antes de `admin/` :

```
path("", include("products.urls")),
```

Transición: demostrar una regla con test rojo-verde.

[F-18] El test rojo revela una regla incumplida

Crea `products/tests/test_models.py` :

```
import pytest
from django.db import IntegrityError
from products.models import Category, Product, ProductVariant

@pytest.mark.django_db
def test_variant_rejects_duplicate_product_size_color():
    category = Category.objects.create(name="Remeras", slug="remeras")
    product = Product.objects.create(
        category=category, name="Clásica", slug="clasica", description="Remera"
    )
    ProductVariant.objects.create(
        product=product, size="M", color="Negro", sku="REM-M-NEG", price=100, stock=2
    )

    with pytest.raises(IntegrityError):
        ProductVariant.objects.create(
            product=product, size="M", color="Negro", sku="OTRO", price=100, stock=2
        )
```

```
pytest products/tests/test_models.py -q
```

Transición: transformar relaciones en un flujo operativo usable.

[F-19] Los inlines convierten relaciones en flujo operativo

Reemplaza `products/admin.py` :

```

from django.contrib import admin
from .models import Category, Product, ProductImage, ProductVariant

class ProductVariantInline(admin.TabularInline):
    model = ProductVariant
    extra = 1
    fields = ("size", "color", "sku", "price", "stock", "active")

class ProductImageInline(admin.TabularInline):
    model = ProductImage
    extra = 1
    fields = ("image", "alt_text", "primary")

@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ("name", "category", "active", "featured", "created_at")
    list_filter = ("active", "featured", "category")
    search_fields = ("name", "brand", "variants__sku")
    prepopulated_fields = {"slug": ("name",)}
    inlines = [ProductVariantInline, ProductImageInline]
    actions = ["publish_selected"]

    @admin.action(description="Publicar productos seleccionados", permissions=["publish"])
    def publish_selected(self, request, queryset):
        queryset.update(active=True)

    def has_publish_permission(self, request):
        return request.user.has_perm("products.publish_product")

admin.site.register(Category)

```

Asigna permisos al grupo:

```

from django.contrib.auth.models import Group, Permission

group = Group.objects.get(name="Operadores")
permissions = Permission.objects.filter(
    content_type__app_label="products",
    codename__in=[
        "view_category", "add_category", "change_category",
        "view_product", "add_product", "change_product", "publish_product",
        "view_productvariant", "add_productvariant", "change_productvariant",
        "view_productimage", "add_productimage", "change_productimage",
    ],
)
group.permissions.set(permissions)

```

Transición: cargar un producto completo.

[F-20] Cargamos productos que cuentan una historia real

Qué realiza el profesor: entra como operador y carga:

- categoría Remeras ;
- producto Remera clásica ;
- imagen y texto alternativo;
- variantes Negro/S, Negro/M y Blanco/M;
- una variante sin stock.

Qué señalar: el operador no ve Users ni Groups y no puede borrar productos.

Cierre posible: si termina la sesión, guardar `c05-admin-data` y continuar desde allí en la próxima clase.

Transición: conectar MiniStore al ORM.

[F-21] MiniStore aporta presentación, no lógica

Qué realiza el profesor: compara `ministore-original.html` con `templates/store/base.html`.

Transformaciones a remarcar:

```
{% load static %}
<link rel="stylesheet" href="{% static 'store/style.css' %}">
<a href="{% url 'products:list' %}">Productos</a>
```

Transición: entregar contexto real mediante CBV.

[F-22] Las CBV entregan contexto real a los templates

Crea `products/views.py`:

```

from django.db.models import Prefetch
from django.views.generic import DetailView, ListView, TemplateView
from .models import Product, ProductVariant

class ActiveProductsMixin:
    def active_products(self):
        active_variants = ProductVariant.objects.filter(active=True)
        return (
            Product.objects.filter(active=True)
            .select_related("category")
            .prefetch_related("images", Prefetch("variants", queryset=active_variants))
        )

class HomeView(ActiveProductsMixin, TemplateView):
    template_name = "products/home.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["featured_products"] = self.active_products().filter(featured=True)[:4]
        context["new_products"] = self.active_products()[:8]
        return context

class ProductListView(ActiveProductsMixin, ListView):
    template_name = "products/product_list.html"
    context_object_name = "products"

    def get_queryset(self):
        return self.active_products()

class ProductDetailView(ActiveProductsMixin, DetailView):
    template_name = "products/product_detail.html"
    context_object_name = "product"
    slug_field = "slug"
    slug_url_kwarg = "slug"

    def get_queryset(self):
        return self.active_products()

```

Transición: reemplazar cards fijas por un include.

[F-23] Una card reutilizable reemplaza contenido duplicado

Crea `templates/store/includes/product_card.html` :

```

{% load static %}
<article class="product-card">
    {% with image=product.images.first %}
        {% if image %}
            
        {% else %}
            
        {% endif %}
    {% endwith %}
    <h3 class="h5 mt-3">
        <a href="{{ product.get_absolute_url }}">{{ product.name }}</a>
    </h3>
    <p>{{ product.category }}</p>
    {% if product.starting_price %}
        <strong>Desde ${{ product.starting_price }}</strong>
    {% else %}
        <span>No disponible</span>
    {% endif %}
</article>

```

Crea `products/templates/products/product_list.html` :

```
{% extends "store/base.html" %}
{% load static %}
{% block content %}
<section class="container py-5">
  <h1>Productos</h1>
  <div class="row g-4">
    {% for product in products %}
      <div class="col-md-4">{% include "store/includes/product_card.html" %}</div>
    {% empty %}
      <p>Todavía no hay productos disponibles.</p>
    {% endfor %}
  </div>
</section>
{% endblock %}
```

Transición: mostrar detalle y disponibilidad.

[F-24] El detalle muestra variantes y disponibilidad

Crea `products/templates/products/product_detail.html` :

```
{% extends "store/base.html" %}
{% block content %}
<section class="container py-5">
  <h1>{{ product.name }}</h1>
  <p>{{ product.description }}</p>
  <h2 class="h4">Variantes</h2>
  <ul>
    {% for variant in product.variants.all %}
      <li>
        {{ variant.color }} / {{ variant.size }} - ${{ variant.price }}
        {% if variant.available %}<strong>Disponible</strong>{% else %}<span>Sin stock</span>{% endif %}
      </li>
    {% empty %}
      <li>No hay variantes disponibles.</li>
    {% endfor %}
  </ul>
</section>
{% endblock %}
```

Crea `products/templates/products/home.html` reutilizando cards:

```
{% extends "store/base.html" %}
{% load static %}
{% block content %}
<section class="container py-5">
  <h1>Tienda v0.1</h1>
  <div class="row g-4">
    {% for product in featured_products %}
      <div class="col-md-3">{% include "store/includes/product_card.html" %}</div>
    {% empty %}
      <p>No hay productos destacados.</p>
    {% endfor %}
  </div>
</section>
{% endblock %}
```

Actualiza la ruta raíz para usar `products:home` y elimina la ruta starter cuando el storefront ya funciona.

Transición: proteger el comportamiento con tests de vista.

[F-25] La batería integrada protege Tienda v0.1

Crea `products/tests/test_views.py` :

```
import pytest
from django.urls import reverse
from products.models import Category, Product

@pytest.mark.django_db
def test_product_list_hides_inactive_products(client):
    category = Category.objects.create(name="Remeras", slug="remeras")
    Product.objects.create(
        category=category, name="Visible", slug="visible", description="A", active=True
    )
    Product.objects.create(
        category=category, name="Oculto", slug="oculto", description="B", active=False
    )

    response = client.get(reverse("products:list"))

    assert response.status_code == 200
    assert "Visible" in response.content.decode()
    assert "Oculto" not in response.content.decode()
```

Ejecuta:

```
pytest accounts products -q
```

Resultado esperado: batería mínima completa en verde.

Transición: recorrer el producto sin mirar código.

[F-26] Recorremos la aplicación sin mirar el código

Qué realiza el profesor:

1. visitante abre home, catálogo y detalle;
2. cliente inicia sesión y abre Mi cuenta;
3. operador agrega una variante y confirma que no ve usuarios;
4. administrador abre usuarios y grupos;
5. visitante refresca y ve la variante nueva.

Qué decir: si el recorrido requiere explicar archivos para parecer funcional, todavía no cerramos una versión de producto.

Transición: nombrar la continuidad.

[F-27] Tienda v0.1 queda cerrada antes de extenderla

Resumen docente:

- `accounts` separa identidad;
- `products` separa catálogo;
- Jazzmin personaliza y Django autoriza;
- MiniStore renderiza datos reales mediante templates;
- tests protegen decisiones centrales;
- Tienda v0.1 ya es navegable.

Decisión docente: guardar `c07-v01` . Este checkpoint cierra el incremento obligatorio. Si termina la sesión, la próxima clase continúa con `[F-28]` ; no se comprime el carrito para mostrarlo incompleto.

[F-28] El carrito comienza como estado temporal

Qué realiza el profesor: presenta el bloque opcional y diferencia tres conceptos:

- carrito público: estado temporal asociado a la sesión;
- carrito persistente: modelos asociados al cliente autenticado;
- orden: fotografía comercial confirmada, fuera de este incremento.

Qué enfatizar: "sin persistencia" significa aquí sin modelos comerciales propios ni asociación a un usuario. La sesión permite conservar el carrito entre requests y, según el backend configurado, Django puede almacenar técnicamente esa sesión.

Transición: crear `orders` y comenzar por el comportamiento público.

[F-29] La sesión guarda identificadores, no productos

Qué realiza el profesor:

```
python manage.py startapp orders
```

Agrega `"orders"` a `INSTALLED_APPS` y crea `orders/cart.py` :

```

from decimal import Decimal

from products.models import ProductVariant

CART_SESSION_KEY = "cart"

def add_session_item(request, variant, quantity=1):
    cart = request.session.setdefault(CART_SESSION_KEY, {})
    key = str(variant.pk)
    cart[key] = min(cart.get(key, 0) + quantity, variant.stock)
    request.session.modified = True

def session_cart_rows(request):
    cart = request.session.get(CART_SESSION_KEY, {})
    variants = ProductVariant.objects.filter(
        pk__in=cart.keys(), active=True, product__active=True
    ).select_related("product")

    rows = []
    total = Decimal("0")
    for variant in variants:
        quantity = min(cart[str(variant.pk)], variant.stock)
        subtotal = variant.price * quantity
        rows.append({"variant": variant, "quantity": quantity, "subtotal": subtotal})
        total += subtotal
    return rows, total

```

Qué explicar:

- la sesión guarda claves serializables, no instancias del ORM;
- precio, nombre y stock vuelven a consultarse;
- el carrito expresa intención y no reserva stock.

Transición: conectar ese estado con URLs y vistas.

[F-30] Agregar y ver el carrito completa el flujo público

Qué realiza el profesor: crea `orders/urls.py` .

```

from django.urls import path
from .views import CartAddView, CartDetailView

app_name = "orders"

urlpatterns = [
    path("cart/", CartDetailView.as_view(), name="cart-detail"),
    path("cart/add/<int:variant_id>/", CartAddView.as_view(), name="cart-add"),
]

```

Agrega `path("", include("orders.urls"))` a `config/urls.py` y crea `orders/views.py` :

```

from django.contrib import messages
from django.shortcuts import get_object_or_404, redirect
from django.views import View
from django.views.generic import TemplateView

from products.models import ProductVariant
from .cart import add_session_item, session_cart_rows

class CartAddView(View):
    def post(self, request, variant_id):
        variant = get_object_or_404(
            ProductVariant,
            pk=variant_id,
            active=True,
            stock__gt=0,
            product__active=True,
        )
        add_session_item(request, variant)
        messages.success(request, "Producto agregado al carrito")
        return redirect("orders:cart-detail")

class CartDetailView(TemplateView):
    template_name = "orders/cart_detail.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["cart_rows"], context["cart_total"] = session_cart_rows(self.request)
        return context

```

Crea `orders/templates/orders/cart_detail.html` :

```

{% extends "store/base.html" %}
{% block content %}
<section class="container py-5">
  <h1>Carrito</h1>
  {% for row in cart_rows %}
    <p>{{ row.variant.product.name }} - {{ row.variant }} x {{ row.quantity }}
    <strong>${{ row.subtotal }}</strong>
  </p>
  {% empty %}
    <p>Tu carrito está vacío.</p>
  {% endfor %}
  <p class="h4">Total: ${{ cart_total }}</p>
</section>
{% endblock %}

```

Agrega en el detalle de producto un formulario `POST` por variante disponible:

```

<form method="post" action="{% url 'orders:cart-add' variant.pk %}">
  {% csrf_token %}
  <button class="btn btn-dark" type="submit">Agregar al carrito</button>
</form>

```

Verificación visible: un visitante agrega una variante, recarga y conserva el carrito.

Transición: demostrar que todavía no existe persistencia comercial.

[F-31] Un test demuestra que el carrito público no crea modelos

Qué realiza el profesor: crea `orders/tests/test_session_cart.py` .

```

import pytest
from django.urls import reverse

from products.models import Category, Product, ProductVariant

@pytest.fixture
def variant(db):
    category = Category.objects.create(name="Remeras", slug="remeras")
    product = Product.objects.create(
        category=category, name="Clásica", slug="clasica", description="Remera"
    )
    return ProductVariant.objects.create(
        product=product, size="M", color="Negro",
        sku="REM-M-NEG", price=100, stock=3,
    )

def test_anonymous_cart_is_stored_in_session(client, variant):
    response = client.post(reverse("orders:cart-add", args=[variant.pk]))

    assert response.status_code == 302
    assert client.session["cart"][str(variant.pk)] == 1

```

```
pytest orders/tests/test_session_cart.py -q
```

Qué enfatizar: en este checkpoint `orders` todavía no posee modelos; el test observa la sesión, no tablas de carrito.

Checkpoint posible: guardar `C08-session-cart` .

Transición: incorporar persistencia solamente para clientes autenticados.

[F-32] El cliente autenticado obtiene un carrito persistente

Qué realiza el profesor: crea `orders/models.py` .

```

from django.conf import settings
from django.db import models

class Cart(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        related_name="cart",
        on_delete=models.CASCADE,
    )
    updated_at = models.DateTimeField(auto_now=True)

class CartItem(models.Model):
    cart = models.ForeignKey(Cart, related_name="items", on_delete=models.CASCADE)
    variant = models.ForeignKey("products.ProductVariant", on_delete=models.PROTECT)
    quantity = models.PositiveIntegerField(default=1)

class Meta:
    constraints = [
        models.UniqueConstraint(
            fields=["cart", "variant"],
            name="unique_variant_per_cart",
        )
    ]

```

```
python manage.py makemigrations orders
python manage.py migrate
```

Qué explicar: `PROTECT` evita perder la referencia de un carrito persistente por una baja accidental de variante. El carrito sigue sin ser una orden y no congela precios.

Transición: elegir el almacenamiento según autenticación.

[F-33] Iniciar sesión migra el carrito temporal

Qué realiza el profesor: amplía `orders/cart.py`.

```
from .models import Cart, CartItem

def add_persistent_item(user, variant, quantity=1):
    cart, _ = Cart.objects.get_or_create(user=user)
    item, _ = CartItem.objects.get_or_create(
        cart=cart,
        variant=variant,
        defaults={"quantity": 0},
    )
    item.quantity = min(item.quantity + quantity, variant.stock)
    item.save(update_fields=["quantity"])

def add_item(request, variant, quantity=1):
    if request.user.is_authenticated:
        add_persistent_item(request.user, variant, quantity)
    else:
        add_session_item(request, variant, quantity)

def merge_session_cart(user, session):
    raw_cart = session.get(CART_SESSION_KEY, {})
    variants = ProductVariant.objects.filter(
        pk__in=raw_cart.keys(), active=True, stock__gt=0, product__active=True
    )
    for variant in variants:
        add_persistent_item(user, variant, raw_cart[str(variant.pk)])
    session.pop(CART_SESSION_KEY, None)
    session.modified = True

def cart_rows(request):
    if not request.user.is_authenticated:
        return session_cart_rows(request)

    cart, _ = Cart.objects.get_or_create(user=request.user)
    items = cart.items.select_related("variant", "variant__product")
    rows = [
        {
            "variant": item.variant,
            "quantity": item.quantity,
            "subtotal": item.variant.price * item.quantity,
        }
        for item in items
    ]
    return rows, sum((row["subtotal"] for row in rows), Decimal("0"))
```

Actualiza los imports y llamadas de `orders/views.py`:

```

from .cart import add_item, cart_rows

# En CartAddView.post:
add_item(request, variant)

# En CartDetailView.get_context_data:
context["cart_rows"], context["cart_total"] = cart_rows(self.request)

```

Luego crea `orders/signals.py` :

```

from django.contrib.auth.signals import user_logged_in
from django.dispatch import receiver

from .cart import merge_session_cart

@receiver(user_logged_in)
def migrate_cart_after_login(sender, request, user, **kwargs):
    merge_session_cart(user, request.session)

```

Crea `orders/apps.py` :

```

from django.apps import AppConfig

class OrdersConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "orders"

    def ready(self):
        from . import signals # noqa: F401

```

Reemplaza `"orders"` por `"orders.apps.OrdersConfig"` en `INSTALLED_APPS` .

Qué enfatizar: el signal responde a un evento transversal de autenticación; la lógica de migración permanece en un servicio testeable.

Transición: probar separación y continuidad.

[F-34] Los tests protegen continuidad y separación

Qué realiza el profesor: agrega `orders/tests/test_persistent_cart.py` .

```

import pytest
from django.urls import reverse

from accounts.models import User
from orders.cart import CART_SESSION_KEY, add_item, merge_session_cart
from orders.models import Cart, CartItem

@pytest.mark.django_db
def test_anonymous_cart_does_not_create_persistent_cart(client, variant):
    client.post(reverse("orders:cart-add", args=[variant.pk]))

    assert client.session[CART_SESSION_KEY][str(variant.pk)] == 1
    assert Cart.objects.count() == 0

@pytest.mark.django_db
def test_authenticated_cart_is_persistent(rf, variant):
    user = User.objects.create_user("ana", password="secret123")
    request = rf.post("/")
    request.user = user

    add_item(request, variant)

    assert CartItem.objects.get(cart__user=user, variant=variant).quantity == 1

@pytest.mark.django_db
def test_session_cart_migrates_to_customer(client, variant):
    user = User.objects.create_user("ana", password="secret123")
    session = client.session
    session[CART_SESSION_KEY] = {str(variant.pk): 2}
    session.save()

    merge_session_cart(user, session)

    assert CartItem.objects.get(cart__user=user, variant=variant).quantity == 2
    assert CART_SESSION_KEY not in session

@pytest.mark.django_db
def test_out_of_stock_variant_cannot_be_added(client, variant):
    variant.stock = 0
    variant.save(update_fields=["stock"])

    response = client.post(reverse("orders:cart-add", args=[variant.pk]))

    assert response.status_code == 404
    assert CART_SESSION_KEY not in client.session

```

```

pytest orders -q
pytest accounts products orders -q

```

Nota docente: reutilizar el fixture `variant` mediante `orders/tests/conftest.py` para que ambos módulos de test sean ejecutables juntos.

Resultado esperado: el visitante no crea un carrito persistente; el cliente sí; la migración conserva cantidades válidas.

Transición: recorrer la extensión completa.

[F-35] El carrito prepara la próxima versión

Qué realiza el profesor:

1. abre el catálogo como visitante y agrega una variante;
2. confirma que el carrito se conserva al navegar;
3. inicia sesión como cliente;
4. confirma que el contenido aparece en el carrito persistente;
5. ejecuta la batería completa;
6. guarda `C09-persistent-cart`.

Qué decir: el carrito conserva intención de compra, pero todavía no descuenta stock, no congela precio y no constituye una orden. Esas reglas pertenecen al siguiente incremento.

Cierre: si solo se alcanzó `C07-v01` o `C08-session-cart`, la siguiente clase retoma desde ese checkpoint sin considerar incompleta la versión anterior.

Anexo docente — explicación detallada de todo el código

Este anexo acompaña el live coding sin agregar tareas. Antes o después de escribir cada fragmento, el profesor puede usar la entrada correspondiente para explicar qué resuelve, cómo se ejecuta y qué decisión de diseño representa.

Explicación F-03 — Entorno y comandos iniciales

- `python -m venv .venv` ejecuta el módulo estándar `venv` con el intérprete seleccionado y crea una instalación aislada de Python dentro del proyecto.
- `Activate.ps1` modifica temporalmente `PATH` para que `python` y `pip` apunten a `.venv`. No instala paquetes ni cambia el código.
- `python -m pip` evita ejecutar por error un `pip` perteneciente a otro intérprete.
- `requirements.txt` permite repetir exactamente la preparación del entorno.
- `manage.py check` valida configuración, apps, URLs y modelos sin iniciar el servidor.
- `pytest -q` verifica el estado inicial antes de incorporar cambios.
- `runserver` es un servidor de desarrollo; recarga el código automáticamente y no debe presentarse como servidor de producción.

Explicación F-04 — Creación y registro de `accounts`

- `startapp accounts` crea la estructura convencional de una app, pero no la conecta automáticamente al proyecto.
- Agregar `accounts` a `INSTALLED_APPS` permite que Django descubra modelos, migraciones, templates, configuración y comandos de la app.
- `app_name = "accounts"` define un namespace para evitar colisiones entre nombres de URL.
- `path("profile/", ProfileView.as_view(), name="profile")` asigna una URL estable a la vista basada en clase. `as_view()` convierte la clase en una función invocable por Django.
- En esta etapa la importación de `ProfileView` anticipa el siguiente paso; si aún no existe, se crea una vista mínima antes de ejecutar `check`.

Explicación F-05 — `User`, `CustomerProfile` y primera migración

- `User(AbstractUser)` conserva la implementación de autenticación de Django y deja abierta la posibilidad de extender la identidad en el futuro. Aunque no agregue campos, debe definirse antes de la primera migración porque será el modelo referenciado por el sistema.
- `AUTH_USER_MODEL = "accounts.User"` indica a Django que todas las relaciones de usuario deben apuntar al modelo local y no a `auth.User`.
- `CustomerProfile` separa datos comerciales exclusivos del cliente. Un operador y un administrador siguen siendo usuarios válidos sin tener teléfono ni dirección de entrega.
- `OneToOneField` garantiza como máximo un perfil por usuario. `related_name` permite navegar desde un cliente mediante `user.customer_profile`.
- `on_delete=models.CASCADE` elimina el perfil si se elimina la identidad; nunca elimina al usuario cuando desaparece el perfil.
- `settings.AUTH_USER_MODEL` evita acoplar la relación al nombre concreto de la clase.
- `makemigrations` describe el cambio como operaciones versionadas; `migrate` aplica esas operaciones a la base.
- `LOGIN_URL`, `LOGIN_REDIRECT_URL` y `LOGOUT_REDIRECT_URL` definen el flujo posterior a autenticación sin escribir redirecciones manuales en cada vista.

Explicación F-06 — Vista y template de Mi cuenta

- `LoginRequiredMixin` intercepta usuarios anónimos y los redirige al login.
- `UserPassesTestMixin` ejecuta `test_func()` después de autenticar. La condición `not self.request.user.is_staff` reserva Mi cuenta para clientes y evita crear perfiles comerciales para operadores o administradores.
- `TemplateView` es suficiente porque la pantalla no lista ni edita un modelo directamente.
- `get_context_data()` amplía el contexto estándar; siempre se llama primero a `super()` para conservar los valores que Django ya preparó.
- `get_or_create(user=...)` recupera el perfil del cliente o lo crea en su primer acceso. El resultado es una tupla (objeto, creado); el guion ignora el booleano con `_`.
- El template hereda `store/base.html`, por lo que navegación, estilos y estructura no se duplican.
- `default` presenta un estado comprensible cuando todavía no hay teléfono o dirección.
- El formulario de login usa `POST` porque envía credenciales, y `{% csrf_token %}` protege la petición frente a envíos desde otros sitios.
- `{{ form.as_p }}` muestra el formulario provisto por Django; no implementa autenticación manual ni manipula contraseñas.
- Los tests verifican dos fronteras distintas: anónimo redirigido y usuario staff rechazado con `403`.

Explicación F-07 — Creación de usuarios de demostración

- `createsuperuser` usa el flujo oficial para crear una identidad administrativa.
- `create_user()` aplica el hasher configurado; nunca se asigna una contraseña directamente al campo `password`.
- El cliente se crea sin `is_staff`, y luego recibe explícitamente `CustomerProfile`.
- El operador se crea con `is_staff=True`, condición necesaria para entrar al admin, pero no recibe perfil cliente ni permisos sobre modelos.
- Esta secuencia muestra que tipo de acceso y datos comerciales son decisiones separadas.

Explicación F-08 — Grupo Operadores

- `Group.objects.get_or_create()` vuelve repetible la preparación: recupera el grupo si ya existe o lo crea una sola vez.
- `operator.groups.add(operator_group)` establece la relación muchos-a-muchos sin reemplazar otros grupos.
- El grupo representa un conjunto de permisos reutilizable. No se agrega un campo `role` que duplicaría reglas ya resueltas por Django.
- Los permisos comerciales todavía no se asignan porque sus modelos se crearán junto con `products`.

Explicación F-10 — Test de autorización por URL directa

- `client.force_login(operator)` autentica el cliente de pruebas sin depender del formulario de login; el test queda concentrado en autorización.
- `reverse("admin:accounts_user_changelist")` obtiene la URL por nombre y evita escribir rutas frágiles.
- Solicitar la URL directa demuestra que ocultar un enlace no constituye seguridad.
- El `403` esperado significa que el usuario está autenticado, pero no autorizado.

Explicación F-11 — Configuración de Jazzmin

- `JAZZMIN_SETTINGS.update()` amplía la configuración inicial sin reemplazar opciones ya preparadas.
- `order_with_respect_to` cambia el orden visual de apps y modelos, no sus permisos.
- `icons` asocia nombres `app.Model` con clases visuales; tampoco concede acceso.
- Las referencias a `products` pueden declararse antes de crear la app, pero serán visibles únicamente cuando exista y el usuario tenga permisos.

Explicación F-12 — Registro de `UserAdmin`

- Registrar el `User` personalizado reemplaza el registro que Django realizaría para su modelo predeterminado.
- Heredar `DjangoUserAdmin` conserva formularios seguros, cambio de contraseña, grupos, permisos y campos de estado.
- La clase vacía con `pass` expresa deliberadamente que no se agregan campos comerciales.
- `CustomerProfile` no se registra: el admin gestiona identidades y permisos, mientras el área cliente gestiona teléfono y dirección.

Explicación F-13 — Tests integrados de identidad

- Ejecutar `pytest accounts -q` prueba la app completa y detecta interacciones entre vistas, modelos y autorización.
- El test del cliente crea una identidad sin `is_staff`, inicia sesión y abre Mi cuenta.
- La aserción sobre `customer_profile` confirma el efecto observable de `get_or_create()`.
- Este test complementa, pero no reemplaza, la prueba que rechaza al operador.

Explicación F-14 — Creación y URLs de `products`

- `startapp products` crea una app separada porque catálogo e identidad tienen ciclos de cambio diferentes.
- El namespace `products` permite usar nombres como `products:list` y `products:detail`.
- La ruta vacía representa la portada; `products/` representa el catálogo.
- `<slug:slug>` captura un identificador legible desde la URL y lo entrega a la vista de detalle.
- Importar las tres vistas en un solo módulo mantiene las URLs declarativas: describen navegación y no contienen lógica de consulta.

Explicación F-16 — Modelos del catálogo

- `Category` agrupa productos; `unique=True` en nombre y slug impide categorías ambiguas.
- `Product.category` usa `PROTECT`: una categoría utilizada no puede eliminarse sin resolver primero sus productos.
- `related_name="products"` habilita consultas como `category.products.all()`.
- `Product` contiene datos compartidos por todas las variantes; no guarda stock ni precio.
- `active` controla publicación sin borrar datos, y `featured` selecciona productos para la portada.
- `get_absolute_url()` centraliza la URL canónica del producto para templates y admin.
- `starting_price` consulta la variante activa más económica. Si no existe, devuelve `None` para que el template muestre "No disponible".
- `ProductVariant` representa la combinación vendible. SKU, talla, color, precio y stock pertenecen a esta clase.
- `MinValueValidator(0)` produce errores de validación en formularios; `PositiveIntegerField` expresa que el stock no admite números negativos.
- `UniqueConstraint(product, size, color)` protege una regla que no puede expresarse con un `unique=True` individual.
- `available` combina estado y stock en una propiedad reutilizable.
- `ProductImage` usa `ImageField`, por lo que Pillow valida archivos de imagen; `alt_text` permite describirlos accesiblemente.

Explicación F-17 — Migraciones y relaciones

- `makemigrations products` genera operaciones a partir de los modelos; el archivo resultante forma parte del código versionado.
- `migrate` ejecuta las operaciones pendientes respetando dependencias entre apps.
- Incluir `products.urls` en la raíz delega la navegación pública a la app.
- El orden de las rutas importa: Django evalúa patrones de arriba hacia abajo.
- `PROTECT`, `CASCADE` y restricciones se explican como reglas persistentes, no como detalles visuales del admin.

Explicación F-18 — Test de unicidad de variante

- El test crea todos sus datos y no depende de la base usada durante la demostración.
- La primera variante establece la combinación que debe ser única.
- `pytest.raises(IntegrityError)` afirma que la base rechaza la segunda inserción.
- El SKU alternativo demuestra que el fallo proviene de producto+talle+color y no de repetir el SKU.
- La prueba debe ejecutarse con `django_db` porque escribe y verifica una restricción real.

Explicación F-19 — Admin del catálogo y permisos

- Los inlines permiten editar relaciones hijas desde el formulario del producto.
- `TabularInline` prioriza una tabla compacta; `fields` controla exactamente qué columnas ve el operador.
- `list_display`, `list_filter` y `search_fields` convierten el listado en una herramienta operativa.
- `prepopulated_fields` propone el slug desde el nombre; el valor sigue siendo editable.
- La acción `publish_selected` modifica un queryset completo y exige el permiso personalizado mediante `permissions=["publish"]`.
- `has_publish_permission()` conecta el nombre corto `publish` con `products.publish_product`.
- `Permission.objects.filter(content_type__app_label="products", codename__in=...)` limita la asignación a permisos concretos de la app.
- No incluir permisos `delete` impide el borrado, pero no reemplaza validaciones de negocio.
- `group.permissions.set()` deja el conjunto exactamente como fue definido; es adecuado para una preparación controlada de la demostración.

Explicación F-21 — Adaptación de MiniStore

- `{% load static %}` habilita la etiqueta que resuelve assets según la configuración de Django.
- `{% static 'store/style.css' %}` reemplaza rutas relativas dependientes del archivo HTML.
- `{% url 'products:list' %}` resuelve navegación por nombre y evita acoplarla a una cadena.
- La adaptación conserva clases Bootstrap y estructura visual; Django reemplaza datos y navegación estáticos por valores del servidor.

Explicación F-22 — CBV y consultas del storefront

- `ActiveProductsMixin` concentra la definición de “producto visible” para no repetir filtros.
- `select_related("category")` resuelve la FK mediante un join y evita una consulta adicional por producto.
- `Prefetch` permite filtrar las variantes activas antes de cargarlas en memoria.
- `prefetch_related("images", ...)` resuelve relaciones múltiples con consultas separadas y evita el problema N+1 al renderizar cards.
- `HomeView` hereda `TemplateView` porque arma varias colecciones con nombres distintos.
- `get_context_data()` agrega destacados y novedades al contexto.
- `ProductListView` usa `context_object_name` para que el template lea `products`.
- `ProductDetailView` busca por slug y reutiliza el mismo queryset visible, por lo que un producto inactivo produce `404`.
- Limitar con `[:4]` y `[:8]` ocurre en SQL cuando el queryset se evalúa.

Explicación F-23 — Include de producto y catálogo

- El include recibe automáticamente el contexto actual, incluido `product`.
- `{% with image=product.images.first %}` evita repetir la expresión y permite resolver un fallback.
- El texto alternativo proviene del modelo; la imagen decorativa fallback usa `alt=""`.
- `product.get_absolute_url` delega la construcción del enlace al modelo.
- La condición sobre `starting_price` diferencia productos vendibles de productos sin variantes activas.
- El loop del catálogo reutiliza la card; `{% empty %}` cubre explícitamente el catálogo sin resultados.
- Las columnas Bootstrap modifican disposición visual sin alterar la consulta.

Explicación F-24 — Detalle y portada

- El detalle itera las variantes ya asociadas al producto y usa `available` para representar stock y estado con una sola regla.
- El bloque `{% empty %}` evita una lista vacía sin explicación.
- La portada reutiliza el mismo include que el catálogo; esto mantiene nombre, imagen y precio consistentes.
- Cambiar la ruta raíz al final evita romper la demostración antes de que `products:home` esté listo.

Explicación F-25 — Test del catálogo público

- El test crea un producto activo y otro inactivo con datos mínimos explícitos.
- `reverse("products:list")` prueba el contrato público de URL.
- Verificar `status_code == 200` confirma que la vista respondió correctamente.
- Las aserciones sobre contenido demuestran la regla funcional: visible aparece e inactivo no se filtra solamente en el template, sino desde el queryset.
- Ejecutar `pytest accounts products -q` comprueba que catálogo e identidad continúan funcionando juntos.

Explicación F-29 — Servicio de carrito en sesión

- `CART_SESSION_KEY` evita repetir una cadena y define un único lugar para cambiar la clave.
- `request.session.setdefault()` recupera el diccionario existente o crea uno vacío.
- Las claves se convierten a `str` porque el serializador JSON de sesiones trabaja con claves de texto de forma predecible.
- `min(..., variant.stock)` impide que la cantidad solicitada exceda el stock observado.
- `request.session.modified = True` obliga a Django a guardar cambios hechos dentro del diccionario anidado.
- `session_cart_rows()` recupera las variantes reales desde el ORM; la sesión no se considera fuente confiable de nombres, precios ni disponibilidad.
- `pk__in`, `active=True` y `product__active=True` descartan referencias inválidas o no publicables.
- `Decimal("0")` conserva aritmética monetaria decimal y evita errores de punto flotante.
- Cada fila reúne variante, cantidad y subtotal para mantener cálculos fuera del template.

Explicación F-30 — Vistas y template del carrito público

- Las URLs separan lectura (`cart/`) de modificación (`cart/add/.../`).
- `CartAddView.post()` no implementa `get()` : agregar una variante es una operación que modifica estado y debe ejecutarse mediante `POST` .
- `get_object_or_404()` combina consulta y respuesta segura; variantes inactivas o sin stock no pueden agregarse.
- El mensaje de éxito sobrevive a la redirección y aporta feedback al usuario.
- Redirigir después del `POST` aplica Post/Redirect/Get y evita repetir la operación al refrescar.
- `CartDetailView` prepara filas y total antes de renderizar.
- El template presenta un estado vacío y no calcula importes.
- El formulario por variante incluye CSRF y envía exactamente la variante elegida.

Explicación F-31 — Test del carrito temporal

- El fixture construye una variante vendible reutilizable.
- El test usa el cliente anónimo: no crea usuario ni inicia sesión.
- Después del `POST`, `client.session` permite observar el estado guardado por la vista.
- La clave textual del ID y la cantidad esperada verifican el contrato interno del carrito.
- En este checkpoint todavía no existen modelos `Cart` ni `CartItem`; la persistencia comercial se incorpora solamente después.

Explicación F-32 — Modelos del carrito persistente

- `Cart.user` es `OneToOneField`: cada cliente conserva un único carrito activo.
- `settings.AUTH_USER_MODEL` respeta el usuario personalizado.
- `updated_at` registra la última modificación útil para soporte o limpieza futura.
- `CartItem.cart` usa `CASCADE` porque los ítems dejan de tener sentido sin el carrito.
- `CartItem.variant` usa `PROTECT` para no romper referencias existentes por una eliminación accidental de catálogo.
- La cantidad es positiva; las operaciones del servicio deben además limitarla por stock.
- La restricción única evita dos filas distintas para la misma variante dentro de un carrito.
- El carrito no congela precios: sigue representando intención, no una orden confirmada.

Explicación F-33 — Selección de almacenamiento y migración al login

- `add_persistent_item()` usa `get_or_create()` para reutilizar carrito e ítem.
- `defaults={"quantity": 0}` permite sumar correctamente la primera unidad; sin ese valor, el default del modelo produciría una cantidad inicial duplicada.
- `add_item()` funciona como punto de entrada único y elige sesión o base según `request.user.is_authenticated`.
- `merge_session_cart()` vuelve a consultar variantes activas y con stock antes de migrar; nunca confía ciegamente en IDs almacenados.
- Después de migrar, elimina la clave temporal para evitar duplicar cantidades.
- `cart_rows()` presenta la misma estructura al template sin importar dónde vive el carrito.
- `select_related` evita consultas adicionales al mostrar variante y producto.
- `sum(..., Decimal("0"))` conserva el tipo decimal incluso cuando el carrito está vacío.
- `user_logged_in` expresa el momento exacto en que una sesión anónima obtiene identidad.
- El receiver delega la lógica a una función testeable; el signal solo conecta el evento.
- `OrdersConfig.ready()` importa los receivers cuando Django termina de cargar apps.

Explicación F-34 — Tests del carrito persistente

- El primer test demuestra separación: un visitante modifica sesión y no crea `Cart` .
- El segundo construye un request controlado con `rf` , asigna un usuario y prueba directamente el servicio persistente.
- El tercero prepara la sesión, ejecuta la migración y comprueba cantidad y limpieza.
- El último coloca stock en cero y verifica que la vista responda `404` sin crear carrito.
- Probar servicios y vistas por separado facilita diagnosticar si falla la regla o la capa HTTP.
- La batería integrada confirma que incorporar `orders` no rompe `accounts` ni `products` .